



A MINOR PROJECT SYNOPSIS

SYNAPSE

(A CNN-Based Handwritten Digit Recognition Web Application)

**Submitted in partial fulfilment of the requirements for the award
of the degree of B.Tech in Computer Science and Engineering**

Submitted By:

Naman Garg - 23BCON0017

Saud Khan - 23BCON0059

Under the Supervision of

Ms. Shraddha Chaudhary

Department of Computer Science & Engineering

Session: 2025-26

JECRC University, Jaipur

TABLE OF CONTENTS

S. No.	Content	Page No.
1	INTRODUCTION	1
2	FEASIBILITY STUDY	2
3	PROJECT FEATURES	3
4	TOOLS, TECHNOLOGIES AND SYSTEM REQUIREMENTS	4
5	WORK PLAN & METHODOLOGY	6
6	CONCLUSION	8

1. INTRODUCTION

Synapse is a full-stack web application that uses a **Convolutional Neural Network (CNN)** to recognize handwritten digits in real time. The system allows authenticated users to either draw a digit directly on an interactive canvas or upload an image file, and then instantly receive the model's predicted digit along with detailed inference statistics.

At its core, **Synapse is powered by a custom-trained CNN model built on the MNIST dataset**, achieving a test **accuracy of 99.3%**. The model uses three convolutional blocks with Batch Normalization, MaxPooling, and Dropout layers, followed by Global Average Pooling and a fully connected head with Softmax output. Data augmentation during training including random rotation, zoom, and translation ensures the model handles real-world handwriting variance effectively.

The backend is developed using Flask (Python), with user authentication handled via Flask-Login and bcrypt password hashing, and all prediction history persisted in a MySQL database through SQLAlchemy. **The frontend features a sleek terminal-inspired UI** with a canvas drawing panel, live inference, softmax heatmap visualization, top-3 candidate display, and a personal prediction history dashboard.

Synapse bridges the gap between academic deep learning and practical deployment demonstrating how a trained neural network can be embedded in a production-ready, user-facing application.

2. FEASIBILITY STUDY

Need:

Handwritten digit recognition has direct real-world applications in postal code reading, bank cheque processing, digitization of historical records, and educational tools. While pre-trained solutions exist, there is a clear need for a user-friendly, web-accessible system where users can interactively test recognition, review history, and understand model behavior through visual statistics.

Significance:

Synapse demonstrates the end-to-end development pipeline for a machine learning application from model training and evaluation to REST API integration and a responsive UI. It serves as both a practical tool and an educational showcase of CNN-based inference in a live web environment.

Feasibility:

- **Technical Feasibility:** The project is built entirely on well-established, freely available open-source technologies: Python, TensorFlow/Keras, Flask, and MySQL. All tools are cross-platform and widely documented.
- **Operational Feasibility:** The web interface is intuitive and requires no special hardware - a standard browser with internet access is sufficient. Both drawing and image upload modes are supported for accessibility.
- **Economic Feasibility:** All software components are free and open-source. The application can be deployed locally or hosted on free-tier platforms such as Render, Railway, or PythonAnywhere with minimal cost.

3. PROJECT FEATURES

Synapse includes the following key features:

- **User Authentication & Account Management:**
 - Secure sign-up and login with bcrypt password hashing
 - Session management via Flask-Login
 - Protected routes - prediction tool accessible only after login
- **Digit Recognition Tool:**
 - Canvas drawing mode with adjustable brush size and real-time auto-prediction
 - Image upload mode supporting PNG, JPG, and BMP formats
 - Predicted digit displayed with confidence percentage and animated fill bar
 - Top-3 candidate digits with probabilities
 - Full Softmax output heatmap across all 10 neuron outputs
- **Prediction History Dashboard:**
 - Personalized grid of all past predictions with canvas snapshots
 - Statistics: total predictions, canvas draws, image uploads, average confidence
 - Individual prediction deletion with animated card removal
- **CNN Model (digit_model_v2.keras):**
 - 3 convolutional blocks with BatchNormalization, MaxPooling, and Dropout
 - Data augmentation: random rotation, zoom, and translation
 - 99.3% accuracy on MNIST test set (10,000 samples)
 - Canvas images preprocessed: grayscale conversion, inversion, resize to 28x28, normalization
- **Additional Features:**
 - Animated preloader with matrix rain effect on every page load
 - Responsive dark-themed terminal-inspired UI
 - Touch/mobile support for canvas drawing

4. TOOLS, TECHNOLOGIES AND SYSTEM REQUIREMENTS

Tools and Technologies Used:

- **Frontend:** HTML5, CSS3, Vanilla JavaScript
- **Backend:** Python 3, Flask 3.x, Flask-Login, Flask-Bcrypt, Flask-SQLAlchemy
- **Machine Learning:** TensorFlow 2.x, Keras, NumPy, OpenCV, Pillow, Matplotlib
- **Database:** MySQL (via PyMySQL), SQLAlchemy ORM
- **IDE:** VS Code
- **Version Control:** Git & GitHub

Hardware Requirements:

- **Processor:** Intel Core i5 or above (for model training)
- **RAM:** 8 GB or higher
- **Storage:** 10 GB free disk space (model + dependencies)
- **Internet Connection:** Required for initial setup

Software Requirements:

- **Operating System:** Windows 10/11, macOS, or Ubuntu
- **Browser:** Chrome, Firefox, or Edge (latest version)
- Python 3.9+ environment with pip
- MySQL Server 8.0 or above

Functional Requirements:

1. Users must be able to register, log in, and log out securely.
2. The system must accept digit input via canvas drawing and image upload.
3. The model must return the predicted digit, confidence score, top-3 candidates, and full Softmax output.
4. All predictions must be saved to the database and viewable on the dashboard.

Non-Functional Requirements:

5. The application should be responsive and accessible on both desktop and mobile browsers.
6. Passwords must be stored as bcrypt hashes; no plaintext credentials should ever be persisted.
7. Inference response time should be under 2 seconds for canvas and upload requests.

5. WORK PLAN & METHODOLOGY

Methodology:

The project follows a modular, iterative development approach, divided into clearly defined phases:

- **Model development**: dataset preparation, CNN architecture design, training with callbacks (ModelCheckpoint, ReduceLROnPlateau, EarlyStopping)
- **Backend setup**: Flask application factory, Blueprints for auth/predict/dashboard routes, SQLAlchemy models
- **Inference pipeline**: image preprocessing (OpenCV), model loading, prediction endpoint with snapshot saving
- **Frontend development**: Jinja2 templating, canvas drawing API, upload UI, result visualization
- **Integration and testing**: end-to-end testing, database verification, cross-browser and mobile compatibility

Project Timeline (5 weeks):

Month	Phase / Task
Week 1 - 2	Requirement Analysis, CNN Architecture Design, Database Schema Design
Week 2 - 3	Model Training: MNIST preprocessing, data augmentation, training with callbacks
Week 3 - 4	Flask Backend: Authentication (Blueprint), prediction routes, SQLAlchemy models
Week 4 - 5	Frontend Development: Canvas UI, upload UI, result panels, dashboard
Week 5	Integration Testing, Bug Fixes, Documentation, Final Submission

Role & Responsibilities:

- **Naman Garg:** CNN model design, training pipeline, preprocessing and inference backend, Flask routes, SQLAlchemy Models, authentication, REST API
- **Saud Khan:** Frontend design, JavaScript canvas/upload logic, dashboard UI, documentation work,

6. CONCLUSION

Synapse is a comprehensive demonstration of how Convolutional Neural Networks can be deployed as real-world, user-facing applications. **By training a custom CNN on the MNIST dataset with modern techniques such as data augmentation, Batch Normalization, and learning rate scheduling, the project achieves a test accuracy of 99.3% -** and exposes that model through a polished, accessible web interface.

The project integrates multiple domains of software engineering: deep learning with TensorFlow/Keras, RESTful backend development with Flask and MySQL, and responsive frontend design with HTML5 Canvas and JavaScript. **The result is a full-stack application that is both technically sound and practically usable.**

Synapse has strong potential for future enhancements, including **multi-digit recognition**, support for other character sets (letters, mathematical symbols), a mobile app version, transfer learning for domain-specific handwriting, and integration of explainable tools such as Grad-CAM to visualize which parts of a drawn digit influenced the model's decision.

This minor project demonstrates that academic machine learning can be meaningfully grounded in real, deployable software - and provides a solid foundation for more advanced AI-powered applications in subsequent semesters.



Naman Garg
(Backend Developer)

Saud khan
(Frontend Developer)

Ms. Shraddha Chaudhary
(Assistant Professor)